Appendix C — ai-engineering-harness — patterns, rate limiting, commands

Source: `README.md`

ai-engineering-harness

Small, runnable LangGraph exercises exploring the same handful of primitives at increasing levels of production-readiness: `Command`, `interrupt()`, and `Send` (dynamic fan-out).

Contents

- **`command.py`, `retry_mechanism_with_command.py`** — `Command` for atomic state-update-plus-routing, and a bounded retry loop pattern.
- **`interrupt_command.py`** → **`approval_service.py`** — the same human-in-the-loop approval gate, first as a single in-process script (`MemorySaver`), then wrapped in a FastAPI service with a durable (`SqliteSaver`) checkpointer so the pause/resume can span separate HTTP requests.
- **`scratch_1_.py`, `send.py`** → **`send_tavily.py`** / **`send_tavily_truncate.py`** — dynamic fan-out with `Send`, from a stub two-branch example up to a real parallel web-search-and-synthesize graph (Tavily + OpenAI), hardened with:
- per-branch failure isolation (a failed search doesn't crash the whole run)
- a bounded fan-out width with two alternative strategies — reject (`send_tavily.py`) vs. truncate-with-a-note (`send_tavily_truncate.py`)
- untrusted-content fencing around retrieved web content before it enters the model's context (indirect prompt-injection mitigation)
- structured output with citation validation (claimed sources are checked against what was actually retrieved)
- async node functions + `ainvoke` for genuine concurrent I/O across branches
- **`COMMAND_PATTERNS_REVISION.md`** — condensed reference notes on `Command`.
- **`diagrams/`** — a Mermaid diagram (+ rendered PNG) of the retry pattern.

Running

Each script is self-contained. The `send_tavily*.py` files call the Tavily and OpenAI APIs and expect `TAVILY_API_KEY` / `OPENAI_API_KEY` in the environment (a `.env` file works via `python-dotenv`).

Source: `SYSTEM_DESIGN_PATTERNS.md`

Agentic system design patterns

Four recurring shapes that come up when you move the primitives in this repo (`Command`, `interrupt()`, `Send`, reducers, checkpointing) from a single script into a real multi-agent system operating on business data. Each one follows the same delivery framework: understand the problem → requirements (functional / non-functional) → core entities → API surface → high-level design (one pass per functional requirement) → deep dives on the hard parts → additional considerations (trade-offs, rollout, what's deliberately out of scope).

---

1. A read-only briefing agent (retrieval, no write path)

Understanding the problem

An agent that helps someone prepare for a meeting by pulling relevant context from a CRM-like record store and a team chat tool, and compiling it into a short, cited brief. The record store and chat tool are treated as existing systems with query APIs — out of scope to design; the system under design is the retrieval-and-synthesis layer sitting in front of them.

Functional requirements

Core:

1. A user can request a brief for a given account/deal and receive a synthesized, cited result within a few seconds.
2. The brief surfaces only information the requesting user is already authorized to see — no exceptions, no "usually."

Below the line (out of scope):

- Building the CRM or chat tool themselves.
- Any write-back to the source systems (that's design 3).
- Multi-turn conversational refinement of the brief — v1 is single-shot, request in, brief out.

Non-functional requirements

- **Latency:** interactive, pre-meeting use — an end-to-end budget of a few seconds for retrieval + synthesis, not minutes.
- **Scale:** tens of thousands of users, each requesting a handful of briefs a day, sharply bursty around business hours rather than uniform — the traffic shape looks nothing like a steady-state API.
- **Consistency:** retrieval can tolerate staleness measured in minutes (a deal's fields drift slowly); **access control cannot tolerate any staleness or approximation** — this is a hard requirement, not a tuning knob, and it's the one place in this design where "eventually correct" isn't good enough.
- **Availability:** a single failing retrieval source should degrade the brief (fewer sections, a note about what's missing), not fail the whole request.

The set up

Planning the approach: this is a fan-out retrieval-and-synthesis pipeline sitting behind an authorization boundary. Because nothing in the system writes anywhere, there's no approval-gate machinery to design — the two hard problems are retrieval quality across heterogeneous sources, and making the access-control guarantee structural rather than a filter someone can forget to apply.

Core entities:

- `BriefRequest` — requesting user, target account/deal id, timestamp.
- `RetrievedChunk` — source, content, relevance score, ACL scope.
- `Brief` — compiled text, `citations[]`, generated-at timestamp.
- `AccessGrant` — the (user, account/deal, scope) relationship consulted before any retrieval happens, not after.

API / system interface:

```
POST /briefs
  body: { accountId, requestingUserId }
  returns: { briefId, status: "pending" }        // async — see NFR latency budget

GET /briefs/{briefId}
  returns: { status, brief: { text, citations[] } }
```

High-level design

1. A user requests a brief and receives a synthesized, cited result.

```
request → triage (what kind of prep is this?)
        → parallel retrieve:
            - structured record lookup (CRM fields, deterministic)
            - hybrid search over chat history (BM25 + dense, RRF-fused)
            - knowledge-base retrieval (product/competitive docs)
        → compile: synthesize a brief with inline citations
        → respond
```

The three retrieval branches are independent — no branch's output depends on another's — so they fan out via `Send`, one branch per source, merging into a shared `hits` channel through an additive reducer. Each branch fails in isolation: if chat search times out, the brief still compiles from the other two sources with a note about what's missing, rather than the whole request failing because one of three sources was slow. This is also where the "no HITL gate" design decision belongs — everything downstream of this system is a read, and an approval gate exists to catch consequential, hard-to-reverse actions. A read that only a human ever looks at isn't one; reserving the gate for design 3, where it's load- bearing, is the more precise call than applying it reflexively everywhere.

2. The brief surfaces only authorized information. Access is scoped to records the requesting user already has rights to — territory, ownership, whatever the source system's ACL model is — and that scope is folded into the retrieval **query itself**, before anything is fetched. See Deep Dive 2 for why this has to be a pre-filter and not a post-filter, and why that distinction is a leak, not just an inefficiency.

Deep dives

1. How do we make retrieval both fast and correct across heterogeneous sources?

- **Chunking:** a recursive splitter (paragraph → line → word boundary), not fixed-size — a chunk cut mid-table or mid-heading embeds worse and reads worse once it lands in the compiled brief. Chunk size is a measured choice (recall@k on a golden query set), not a fixed default picked once and forgotten.
- **Hybrid retrieval:** dense embeddings catch "what's the renewal risk" matching a passage about "SOC2 concerns" — semantic, not lexical. They are bad at exact tokens: a deal ID, a ticket number. Sparse (BM25) is the mirror image — exact on rare tokens, blind to synonymy. Fuse the two ranked lists by **rank**, not raw score — Reciprocal Rank Fusion — because BM25 scores and cosine similarities live on incomparable scales, and RRF rewards a document both retrievers agree on over one retriever's confident outlier.
- **Reranking:** a cheap bi-encoder pass gets a candidate set into scope (optimizes recall — don't lose the right passage); an optional cross-encoder rerank on the shortlist buys precision on the results that actually make the cut — worth the added latency when the source pool is large and heterogeneous, skippable when it's already small and ACL-narrowed to a handful of records.
- **Grounding discipline:** every claim in the compiled brief carries a citation back to a specific retrieved chunk. Check claimed sources against what was actually retrieved and strip or flag anything that doesn't match — a fabricated citation is a real failure mode, not a hypothetical one, and the model's self-reported grounding can't be trusted on its own.

2. How do we guarantee zero ACL leakage rather than "usually correct"? Post-filtering (search everything, drop unauthorized results after ranking) is a real leak, not just an inefficiency, for three separate reasons: restricted content still enters whatever reads the candidate set downstream — a reranker, a summarizer — before the filter ever runs; a caller who normally gets five results and suddenly gets two has just learned that three restricted things exist, which is an oracle for enumerating what's hidden; and it relies on application code remembering to filter correctly every single time, which is a bug waiting to happen. Pre-filtering — folding authorization into the retrieval query itself, or better, into the access token the retrieval service was issued — makes leakage impossible by construction rather than prevented by a conditional somewhere in application code.

3. How do we keep retrieval fast under bursty load without serving stale writes? Cache retrieval results on a short TTL (minutes, not hours) — deal state changes, and a stale field in a brief costs more than an extra cache miss. Eviction policy matters more than it looks: plain LRU means one person running a broad scan (pulling every record in a territory for a review) evicts everyone else's warm working set entirely; a scan- resistant policy — LFU with decay, or an admission filter — avoids that specific failure mode where a single cold, one-off access pattern destroys a hot working set that has nothing to do with it.

Additional considerations

- **Eval:** retrieval and generation are evaluated **separately** — recall@k answers whether the right context was even found (a ceiling nothing downstream can fix); precision@k and a groundedness/citation check on the compiled brief answer whether it was used well. Track **leakage rate** (fraction of requests where retrieval touched something the caller shouldn't see) as a release-blocking metric with a target of zero — not a quality metric to trend over time and improve gradually.
- **Why no approval gate:** covered in High-Level Design step 1 — worth restating as an explicit design decision, not an omission.

---

2. A model-routing and governance layer

Understanding the problem

A layer that decides which model handles which task across a portfolio of agents, with cost and audit control. This sits in front of every model call the other agents make — it's cross-cutting infrastructure, not an agent itself.

Functional requirements

Core:

1. Every agent task is routed to a model appropriate to its risk tier.
2. Every model call is logged with enough detail to audit cost, tier, and outcome after the fact.

Below the line (out of scope):

- Building the agents that call through this layer.
- Fine-tuning or training custom models — routing happens across existing providers/tiers only.

Non-functional requirements

- **Cost:** must bound spend from any single task or tenant — a denial-of-wallet attack or a runaway loop should not translate into an unbounded bill.
- **Latency:** the routing decision itself must be near-zero overhead — a lookup plus a rate-limit check, not another model call in the critical path.
- **Availability:** a provider outage on one tier should degrade gracefully (fallback to another tier), not cascade-fail every agent that happens to be routed there.

The set up

Planning the approach: this is a decision-and-enforcement layer, not a new agent, so the design centers on the tiering policy, budget enforcement, and audit logging — not on orchestration complexity.

Core entities:

- `Task` — type, declared risk tier, tenant/team.
- `ModelTier` — name, candidate providers, cost ceiling, fallback tier.
- `Budget` — tenant, tier, remaining tokens, refill rate.
- `CallLog` — task id, tier used, tokens, cost, latency, outcome.

API / system interface:

```
POST /route
  body: { taskType, tenantId, estimatedTokens }
  returns: { model, tier, requestId }

POST /calls/{requestId}/complete
  body: { actualTokens, latencyMs, outcome }
  // writes the audit log entry, reconciles the budget
```

High-level design

1. Every task is routed to a model tier appropriate to its risk.

```
request → classify task risk tier (public-facing copy vs. deal-sensitive analysis vs. write-triggering)
        → route to the model tier mapped to that risk tier
        → execute
        → on provider failure/rate-limit: fall back to the next tier down, logged as a degradation, not
silently
```

The **risk-tier → model-tier mapping** is the actual design artifact here, not the routing code itself — e.g. public marketing copy can go to a smaller, cheaper model with lenient review; anything touching customer-specific sensitive data, or feeding a write-capable downstream agent, goes to the strongest available model with tighter output constraints (structured output, stricter groundedness checks).

2. Every call is auditable after the fact. Every routed call writes a `CallLog` entry — task, tier, model, tokens, cost, latency, outcome — regardless of success or failure, so cost and quality can be reconstructed per-task after the fact rather than only observed live.

Deep dives

1. How do we prevent denial-of-wallet? A per-request rate limiter is the wrong unit for LLM traffic — one request can be 200 tokens, another 200,000. **Token bucket with a variable cost** is the right primitive: `cost = estimated_tokens / 1000` instead of `cost = 1`, so the budget is a genuine token budget, not a request-count budget a single expensive call can blow through unnoticed. Layer a **hard per-tenant/per-task ceiling** on top of the bucket so one workflow can't exhaust the shared budget for everyone else.

2. The routing layer itself runs as many replicas behind a load balancer — how do you enforce one global budget across all of them, without each replica's local limiter effectively multiplying the limit by the replica count? Four options, in increasing order of how well they actually scale:

- **Centralized counter (Redis, atomic `INCR` + `EXPIRE` or a Lua script for token-bucket math).** Every replica checks the same counter, so the limit is exact and global. The cost isn't just latency — it turns Redis into a **hard dependency on the request path**: if it's slow, every request is slow; if it's down, rate limiting is down.
- **Local limiter at `limit / N`.** Zero coordination, zero added latency — each replica just enforces its own slice. Falls apart the moment load isn't evenly spread across replicas: one replica can be wrongly rejecting requests while its siblings sit on unused budget, even though the *system-wide* total is well under the real limit.
- **Distributed token bucket with lease-based refill — the usual production answer.** Each replica periodically checks out a block of tokens from the central bucket (say 50 at once) and spends them locally, re-leasing when low, instead of hitting Redis on every request. This amortizes the network cost over many requests, and the imprecision it introduces is *bounded by the lease size*: a bigger lease means fewer round trips but more slack (more possible over-admission across the fleet before the next lease corrects it); a smaller lease approaches the centralized counter's exactness at the cost of approaching its network overhead too. Lease size is the actual dial between "cheap and approximate" and "expensive and exact" — it's not a separate fourth option, it's a continuum the first two options sit at the ends of.
- **Fail open or fail closed when the centralized piece itself is down** — see below; it's a distinct decision from which of the three enforcement strategies above you pick.

3. Fail open or fail closed when the budget/routing service itself is down? Decided deliberately, and **per tier** — if the budget service is unavailable, does a request proceed anyway (risking an unbounded bill) or get rejected (degrading availability to protect spend)? Fail closed on the expensive tier, fail open on the cheap one: the cost of a short availability hit on cheap traffic is lower than the cost of an unbounded bill on expensive traffic. Worth noticing this compounds with deep dive 2's centralized-counter option specifically: if all 20 replicas share one Redis instance and you fail closed, a single Redis outage becomes a **synchronized full-fleet outage**, not a contained one — every replica fails the same way at the same time, because they all depend on the same thing. A more resilient version keeps a conservative local fallback limiter (option 2's shape) that only activates when the centralized path is detected as down, trading perfect accuracy for partial availability during that window rather than an all-or-nothing failure.

4. How do we detect a provider degradation and route around it before it fully fails? Track latency/error rate per provider and fall back to the tier's backup provider proactively, rather than waiting for hard failures — the same principle as a circuit breaker, applied at the routing layer instead of inside each individual agent.

5. How do we know the tiering policy itself is still correct over time? Track cost and latency **per tier**, not in aggregate — an aggregate number hides whether the expensive tier is being over-used for tasks that didn't need it. Alert on tier-migration drift (tasks classified into a higher tier than their historical baseline) as an early signal of either a classifier regression or a genuine shift in what's being asked of the system.

Additional considerations

- **Trade-off:** a stricter tiering policy costs latency and money on the low end (routing conservatively) or risk on the high end (routing aggressively to save cost). State this explicitly as a deliberate, revisitable policy choice — not a fixed property of the system that was solved once.

---

3. A write agent, gated (the hard one)

Understanding the problem

An agent that updates a record in a system of record — e.g. a deal's status field — based on a conversation in a team chat tool. The moment this agent writes, the risk posture changes completely relative to design 1: everything from the read-only case still applies, plus an entire additional layer that exists only because the action is consequential and not trivially reversible.

Functional requirements

Core:

1. A proposed record change is generated from a conversational trigger, previewed to a human, and applied only after explicit approval.
2. Every applied change is captured in an audit trail sufficient to answer "who approved what, when, from what source" — indefinitely, not just for a retention window.

Below the line (out of scope):

- Auto-approval / no-human-in-the-loop paths for v1 — that's a later, graduated phase (see Additional Considerations), not a day-one requirement.
- Building the record system itself — treated as an existing system with a write API.

Non-functional requirements

- **Exactly-once effect:** an approved write must apply once, even under network retries, a duplicated trigger, or a human double-clicking approve.
- **Durability across a long pause:** an approval decision can take hours; the pending state must survive a process crash and resume correctly, not just survive within one process's lifetime.
- **Consistency of the audit trail specifically:** it must never fork — two different answers to "did this happen" depending on which replica answers is not an acceptable failure mode, even though most of the rest of the system can tolerate looser consistency.

The set up

Planning the approach: this is the one design in the set where the naive version — an agent with a write tool — is actually unsafe. The whole design exists to interpose a durable pause, a human decision, and a deterministic executor between "the model wants to write" and "the write happens."

Core entities:

- `ChangeProposal` — source message, target record, field, proposed value, justification, citation.
- `ApprovalDecision` — proposal id, approver, decision, timestamp.
- `ExecutedChange` — proposal id, applied value, applied-at, verified (bool — did the read-back confirm it landed).
- `AuditEntry` — immutable, append-only, correlating proposal + decision
- execution into one record.

API / system interface:

```
POST /proposals                          // triggered internally by the reasoning agent
  body: { sourceMessageId, targetRecordId, field, proposedValue, justification }
  returns: { proposalId, status: "pending_approval" }

POST /proposals/{proposalId}/decision
  body: { approverId, decision: "approve" | "reject" }
  // on approve, triggers the deterministic executor

GET /proposals/{proposalId}
  returns: full status including the correlated audit trail
```

High-level design

1. A proposed change is generated and held for approval.

```
trigger (chat message) → reasoning agent proposes a change
                           (field, new value, justification, citation to the source message)
                       → interrupt(): pause, surface a preview of the exact diff to a human
```

`interrupt()` pauses execution and checkpoints exactly what's pending — the proposed diff, the source citation — so the approval can happen asynchronously, potentially hours later, without holding a process open.

2. On approval, the change is applied exactly once by a separate, deterministic executor.

```
on approval: deterministic executor performs the write (not the reasoning agent itself)
           → read-back verification: re-fetch the record, confirm the write landed as intended
```

Reasoning and execution are architecturally separate nodes. The LLM that proposed the change never holds write credentials — a second, deterministic (non-LLM) executor performs the actual API call, with exactly the previously-approved payload and nothing else. This is also the actual defense against a prompt-injection payload hidden in the source chat message: such a payload can at most influence what gets *proposed* for review; it cannot reach the write path on its own, because the executor only ever executes what a human explicitly approved, verbatim.

3. Every change is auditable.

```
→ append to an immutable audit trail (who approved, what changed, when, source citation)
```

See Deep Dive 4 for why this specific component has a stricter consistency requirement than everything else in the design.

Deep dives

1. How do we pause for an approval that might take hours, without holding a process open? `interrupt()` plus a durable checkpointer — the paused state (the proposal, its inputs) is persisted, and the process can shut down entirely; resuming later reloads that checkpoint and continues from there rather than requiring a live, blocked process the whole time.

2. What's the replay gotcha, and how do we design around it? On resume, the node that called `interrupt()` re-runs **from its own top**, not from the middle — so any side effect placed *before* the `interrupt()` call in that same node executes twice: once leading into the pause, once again on replay. The fix is structural: put `interrupt()` as early as possible in the node, before any side-effecting code, so there's nothing upstream of it left to duplicate.

3. How do we guarantee exactly-once application under retries or duplicate triggers? Make the write itself idempotent, independent of the interrupt mechanics. Derive an idempotency key from the *intent* of the write (who, what field, what value, what source event) rather than from the delivery mechanism, so two independently-triggered attempts at the same real change collapse to one. The check-and-claim step must itself be a single atomic operation (`INSERT ... ON CONFLICT DO NOTHING` or equivalent) — a naive check-then-write is itself a race between two concurrent attempts, and would just relocate the exact bug it's meant to fix.

4. Why must the audit trail specifically be the one strongly consistent component, when the rest of the system tolerates staleness fine? Everything upstream of it (retrieval, drafting) can tolerate staleness — a slightly-behind cache is cheap. The record of "did this write happen, and was it approved" cannot fork: two different answers to that question depending on which replica you ask defeats the entire purpose of having an audit trail. In

practice this means backing it with a single linearizable store, even while the rest of the system is built for availability over strict consistency — the consistency requirement is scoped to the smallest component that actually needs it, not applied uniformly across the whole system.

Additional considerations

- **Graduated trust rollout:** full human approval on every write during pilot, moving to sampled/spot-check approval or threshold-based auto-approval only once the human-override rate has been low and stable for a defined observation window. State the threshold and the window explicitly — "we'll loosen it when it feels safe" isn't a design, it's a deferral.
- **Eval:** the human-override rate (how often a human rejects or edits the proposed change) is a leading trust signal here that a purely read-only agent doesn't get — it surfaces model unreliability before it ever shows up in a downstream metric.

---

4. Trust evaluation for pilot → general rollout

Understanding the problem

Given a set of agents already running in a limited pilot — including designs 1 through 3 above — how do you decide they're trustworthy enough to expand to the full organization? This is a process and measurement design, not a service with its own API — but it follows the same discipline: define what "trustworthy enough" means measurably, before the rollout decision has to be made under pressure.

Functional requirements

Core:

1. There is a quantitative, pre-committed bar an agent must clear before expansion — not a subjective "it feels ready" call made after seeing the pilot's numbers.
2. Expansion happens in reversible, bounded increments, with a defined incident/rollback path, not as a single global switch-flip.

Below the line (out of scope):

- The agents' own functional design (covered in 1–3).
- Automating the rollback mechanism itself — assumed to exist as standard feature-flag infrastructure.

Non-functional requirements

- **Auditability of the decision itself:** the rollout decision must be traceable to specific, recorded metrics — not made and then rationalized after the fact.
- **Bounded blast radius:** a bad expansion must be containable to the segment it started in.

The set up

Core entities:

- `GoldenTask` — a held-out, human-labeled task, fixed once the evaluation period starts (a moving target defeats the point of a benchmark).
- `ShadowRun` — the agent's output on live input, logged but not acted on, compared against what a human actually decided.
- `OverrideEvent` — a human rejecting or editing an agent's proposal, tracked over time, not just at a single pilot checkpoint.

- `RolloutSegment` — a team or territory behind its own feature flag, the unit of both expansion and rollback.

High-level design

The rollout pipeline, as a sequence of gates rather than a single decision:

1. **Golden-set task success rate** measured against the fixed, held-out set, checked against a threshold set *before* rollout begins.
2. **Shadow-mode period** — before the agent's proposals affect anything real, run it in parallel against live input with output logged but not acted on. This catches a class of failure a static golden set can't: genuine production input distribution, not a curated sample.
3. **Human-override rate**, tracked as an ongoing signal, not just a one-time gate — trending it after launch catches silent model or data drift that a pre-launch check can't.
4. **Leakage rate held at zero**, carried over from design 1 — a release-blocker at every rollout stage, not something that relaxes as trust in functional quality grows.
5. **Phased, flagged expansion by segment**, with an incident/rollback plan defined *before* expansion, not improvised during an incident — a bad rollout is contained to the segment it started in and rolled back without a full system outage.

Deep dives

1. Why isn't golden-set accuracy enough on its own? A model can pass every golden-set task and still have an override rate that says people don't trust its judgment on the long tail of real input — the golden set, by construction, never contains that tail. This is the actual gap the rest of the framework exists to close.

2. Why shadow mode instead of just a bigger golden set? A golden set is static and curated; shadow mode measures the agent against the real, messy, live input distribution — including inputs nobody thought to label — without any risk, since nothing is acted on yet.

3. Why segment the rollout instead of a single global flag? Bounding blast radius: if the override rate spikes or the golden-set performance doesn't hold in production, the damage and the rollback are both scoped to one segment, not the whole organization.

Additional considerations

- **The judgment call this framework is actually testing:** functional accuracy and trustworthiness are different measurements, and treating a high golden-set score as sufficient evidence for full rollout is the most common mistake this framework is designed to prevent.

---

5. Caching, situationally

Caching rarely shows up as its own design problem here — it shows up as a follow-up inside design 1 or 2 once a cache is on the whiteboard. The questions below are the shapes that follow-up actually takes.

1. How do we keep a cache from serving stale data after the underlying record is updated? Invalidate (delete the key) on write, don't update it in place. An in-place update races with a concurrent reader who fetched the old DB value before the write and caches it *after* the write completes — leaving stale data cached with no TTL boundary to eventually correct it. Deleting is idempotent under that race: the next read simply misses and repopulates from the source of truth.

2. What happens when a popular key expires and hundreds of requests land on it in the same instant? A cache stampede — every one of them misses and every one of them hits the database at once, often at exactly the moment popularity makes that worst. Two standard fixes: a per-key lock so only one caller repopulates while the

rest wait (or get served stale data during the window), or probabilistic early expiry — refresh a key slightly before its TTL, with a probability that rises as the key ages, so refreshes spread out instead of synchronizing on the expiry instant.

3. Local cache per replica, or one centralized cache shared by the fleet? Local is zero-hop and fast, but every replica starts cold independently and two replicas can disagree about the same key at the same moment — fine for pure performance, risky for anything where staleness has a real cost. Centralized (e.g. Redis) gives the whole fleet one consistent view at the cost of a network hop per request and a new shared dependency. Neither is a universal answer — the choice tracks how expensive disagreement between replicas actually is for that specific cache.

4. What stops a single bulk scan from evicting everyone else's warm working set? Plain LRU treats every scanned item as "most recently used," so a one-off sequential scan of cold data (a bulk export, a manager pulling every record in a territory) evicts the entire hot working set on its way through — even though none of the evicted items had anything to do with the scan. A scan-resistant eviction policy — LFU with decay, or an admission filter (e.g. TinyLFU) — only lets a key stay if it's actually been accessed repeatedly, so a scanned-once key is evicted immediately without displacing anything hot.

5. How do we actually choose a TTL, rather than picking a number? It's a cost trade-off, not a constant: weigh the cost of serving stale data against the cost of an extra cache miss, for that specific entity. Fast-changing, consequential data (a deal's live status) gets a short TTL — minutes — because a stale read costs more than a miss. Slow- changing, low-stakes data can tolerate a much longer one. Be able to name what would change the number, not just defend the number itself.

6. How does cache locality interact with load balancing on LLM-serving infrastructure specifically? Prompt/prefix caching (provider-side prompt caching, or KV-cache reuse on self-hosted inference) works by keeping a shared prompt prefix's attention state resident so a later request with the same prefix skips recomputing it — but that cache is local to whichever server instance built it. Round-robin load balancing scatters repeated requests for the same context across every replica, so each one rebuilds the prefix from scratch: a caching feature paid for and barely used, roughly a `1/N` hit rate. The fix is sticky routing — consistent-hash on a key that matches the cache key (e.g. `tenant:promptVersion:entityId`) so repeated requests for the same context land on the replica that already holds it, and because the routing is consistent-hashed, adding a replica under load only moves a small fraction of keys instead of invalidating every warm prefix in the fleet. The honest trade-off to name: sticky routing gives up perfect load balance — a hot key pins load onto one node — and the production compromise is bounded-load consistent hashing: route to the preferred node, but spill to the next one clockwise if it's over a load threshold.

7. Can a cache leak data across users, and how do we prevent it? Yes, if the cache key isn't scoped tightly enough. A semantically-cached LLM response or a shared retrieval cache keyed only on the query text — not on `(user, query)` or `(tenant, query)` — can serve one user's cached answer, built from data only they were authorized to see, straight to a different user asking something similar. This is the same underlying failure as ACL leakage through post-filtered retrieval (design 1, deep dive 2), just relocated into the cache key instead of the query path — the fix is the same principle: authorization has to be part of what identifies the cached entry, not bolted on after a hit.

8. What's the failure mode if the cache goes down entirely, and which do we choose? The same fail-open/fail-closed judgment call as the rate limiter and the model-routing budget service (design 2): fail open falls through to the database directly — slower, but still correct; treating the cache as a hard dependency turns a cache outage into a full outage. State the choice explicitly per cache rather than defaulting into whichever one the framework happens to do automatically.

Source: `RATE_LIMITING_AGENTIC_SYSTEMS.md`

Rate limiting for agentic systems

Rate limiting is usually taught as "protect an API from too many requests." That framing undersells what it actually has to do once the thing making requests is an **agent** rather than a human clicking a button: one top-level trigger can fan out into dozens of LLM calls, tool calls, and retries; the cost of a single call isn't known until it's finished streaming; and a bug in the agent's own control flow can generate more load in one run than a thousand normal users would in a day. This doc covers the five core algorithms in full, then the parts that are specific to agentic systems — multi-layer enforcement, unknown-cost-upfront requests, runaway-loop protection, and protecting fragile things downstream — with references to the runnable examples already in this repo (`send_tavily.py`) where the pattern is actually implemented.

Part 1: why agentic systems break the classic model

A classic API rate limiter assumes: one inbound request ≈ one unit of work, of roughly known and bounded cost, initiated by something with human-scale patience for a 429. None of those three assumptions survive contact with an agent:

1. **One trigger, unbounded fan-out.** A single user request ("summarize this account") can become a triage call, three parallel retrieval calls, a synthesis call, and — if anything in that chain retries or a planning loop decides it needs more evidence — an arbitrary number of further calls. Rate limiting the *inbound* request tells you nothing about the *actual* load it generates.
2. **Cost isn't known until the call finishes.** An LLM call's true cost is a function of how many tokens it actually generates, which you don't know until generation completes — especially with streaming. A classic limiter that charges a flat `cost=1` per request is measuring the wrong thing entirely.
3. **A stuck loop is not "too many users," it's one broken run.** A retry-until-success node, or a planner that keeps deciding it needs "just one more" piece of evidence, can generate more requests in ten seconds than your entire real user base generates in an hour — and no per-user or per-IP limiter was designed to catch a single run misbehaving against itself.

The rest of this doc is organized around fixing each of these three assumptions in turn.

Part 2: the five core algorithms, in full

Every rate limiter answers the same question — "should this unit of work be admitted right now?" — with a different trade-off between exactness, memory, and how it handles bursts.

1. Token bucket

A bucket holds up to `capacity` tokens and refills continuously at `rate` tokens/second. A request is admitted if enough tokens are available.

```
def allow(self, cost=1.0):
    now = self.clock.now()
    self.tokens = min(self.capacity, self.tokens + (now - self.last) * self.rate)
    self.last = now
    if self.tokens >= cost:
        self.tokens -= cost
        return True
    return False
```

Property: permits a burst up to `capacity` (an idle client "saves up"), then throttles to the steady `rate`. **The property that matters most for LLM traffic specifically:** `cost` doesn't have to be `1` — charging `cost = estimated_tokens / 1000` turns this into an actual *token* budget instead of a request-count budget, which is the correct unit when one call is 200 tokens and the next is 200,000. This is the standard answer to "how do you prevent denial-of-wallet."

Verified, `capacity=5, rate=1/s`:

```
8 instant requests: [True, True, True, True, True, False, False, False]
after advancing 3s: [True, True, True, False]     # exactly 3 tokens refilled
```

2. Leaky bucket

Requests enter a fixed-size queue that drains ("leaks") at a **constant** rate, regardless of how they arrived.

```
def allow(self, cost=1.0):
    now = self.clock.now()
    self.level = max(0.0, self.level - (now - self.last) * self.leak_rate)
    self.last = now
    if self.level + cost <= self.capacity:
        self.level += cost
        return True
    return False
```

Property: output is perfectly smoothed — bursts never reach whatever's downstream. **Where it earns its place over token bucket:** token bucket protects *you* from a bursty caller; leaky bucket protects **the thing behind you** — a third-party API with a hard per-second cap that will 429 you (or worse) the instant you burst at it, no matter how briefly. If your agent fans out five parallel searches against a provider with a strict per-second ceiling, leaky-bucket-style pacing on the *outbound* side is what keeps that fan-out from tripping the provider's own limiter.

Verified, same parameters: `[True ×5, False ×3]` — identical admission pattern to token bucket for a burst from empty, but the *shaping* differs under sustained load: leaky bucket paces admission at exactly `leak_rate`; token bucket lets a full-capacity burst through instantaneously the moment it's refilled.

3. Fixed window counter

Count requests per fixed calendar window; reset at the boundary.

Property: O(1) memory, trivial to implement. **The flaw, and it's serious:** the counter has no memory of the previous window, so a client can send `limit` requests at the tail of one window and `limit` more at the head of the next — both individually within-limit, but landing within a fraction of a second of each other in real time.

Verified, `limit=100 per 60s`:

```
100 allowed at t=59s, 100 at t=60s -> 200 in a 1.0s span. 2x the intended rate.
```

4. Sliding window log

Store a timestamp for every allowed request; on each new request, evict timestamps older than the window and count what remains.

Property: exact. No boundary burst, no approximation — the window is always "the last N seconds, ending right now," recalculated fresh on every call, with no fixed calendar alignment to exploit.

Verified on the same adversarial pattern that broke fixed window:

```
100 at t=59s, 0 at t=60s -> 100 total. Boundary burst eliminated.
```

The cost: O(limit) memory *per key*. A limit of 10,000/minute across 1,000 tenants is up to 10 million timestamps resident at once — at scale this becomes the dominant cost, which is why it's usually not what actually runs in production.

5. Sliding window counter — the production default

The compromise: keep only **two** counters per key — the current window's count and the previous window's count — and estimate the true rolling count by weighting the previous window by how much of it still overlaps the sliding N-second lookback:

```
elapsed  = (now mod window) / window
estimate = prev_count * (1 - elapsed) + cur_count
```

Walk the intuition: `elapsed` is "how far into the current window are we, as a fraction" — 0.0 right at the start of the window, approaching 1.0 right before it rolls over. `(1 - elapsed)` is therefore "how much of the *previous* window's traffic should still count as if it happened within the last N seconds." Right at the start of a new window (`elapsed≈0`), almost all of the previous window's count still applies — which is exactly what should happen, since almost the entire sliding lookback still overlaps the previous window at that instant. By the time you're halfway through the current window (`elapsed=0.5`), only half of the previous window's count is still "in view."

Verified:

```
100 at t=59s, 0 at t=60s -> 100 total     (at t=60, elapsed≈0, so prev counts almost fully)
at t=90s (halfway into window 1): another 50 allowed
                                  - weight on the previous window has decayed to ~0.5
```

Property: O(1) memory, closes the boundary-burst hole from fixed window, and is only an *approximation* — it assumes requests were uniformly distributed within the previous window, so it can be slightly wrong if that window was itself bursty. In practice the error is small and bounded, which is why this — not the exact sliding log — is what most production rate limiters actually run.

Comparison

Algorithm	Memory/ key	Bursts	Exact	Boundary burst	Smooths output	Use when
Token bucket	O(1)	Allowed up to capacity	Yes	No	No	Variable-cost LLM token budgets
Leaky bucket	O(1)	Absorbed into the queue	Yes	No	**Yes**	Protecting a fragile downstream with a hard cap
Fixed window	O(1)	Yes, and at the boundary	Yes within a window	**Yes — 2× limit**	No	Coarse quotas where a burst is tolerable
Sliding window log	**O(limit)**	No	**Yes**	No	No	Low-cardinality keys, strict correctness
Sliding window counter	O(1)	Bounded	Approximate	No	No	**The production default** at high cardinality

---

Part 3: multi-layer enforcement — the core agentic-system difference

A single limiter at the API boundary answers "is this inbound request allowed?" — it says nothing about what that request then does internally. An agentic system needs limits at **every layer a fan-out can happen**, because a request that passes the outer limiter can still generate unbounded internal load:

```
per-user / per-session request rate     (classic API layer — coarse, cheap, first line of defense)
  → per-thread / per-run token budget    (a token-bucket, variable cost, scoped to one agent run)
    → per-node / per-tool-call ceiling   (structural, not time-based — see Part 5)
      → per-tool-type budget             (a misbehaving single tool shouldn't hide inside aggregate
headroom)
        → per-provider rate limit        (leaky-bucket-shaped, protecting what's actually downstream)
```

Each layer catches a different failure mode. The outer layer catches a human or script hammering your API. The per-run budget catches a single expensive conversation before it becomes an expensive habit. The structural fan-out ceiling (next section) catches a planning bug, not a volume problem at all. The per-tool budget catches one tool's outage or misbehavior from being masked by the fact that the *other* nine tools still have headroom. The provider-level limiter protects the one thing you don't control the failure mode of.

The design mistake to avoid: treating "we have a rate limiter" as one answer. The question "where" is doing as much work as the question "which algorithm" — a system with only the outer layer looks fine under load testing with well-behaved clients and then falls over the first time an internal loop misbehaves, because nothing downstream of the entry point was ever bounded at all.

Part 4: structural ceilings — bounding a single run, not traffic over time

Everything in Part 2 bounds requests **over time**. A stuck planning loop or a pathological fan-out isn't a time-based problem — it can happen entirely within one run, in seconds, and a rate limiter tuned for sustained abuse won't even notice it happening, because the aggregate rate might still look normal system-wide.

This is a genuinely different mechanism: a **hard, structural ceiling on fan-out width or step count within a single run**, independent of anything time-based. This repo already has one, in `send_tavily.py`:

```
MAX_QUERIES = 5

def fan_out(state: FanState):
    list_length = len(state["queries"])
    if list_length > MAX_QUERIES:
        raise ValueError(f"The total fan out is {list_length} which exceeds {MAX_QUERIES}")
    return [Send("retrieve", {"query": q}) for q in state["queries"]]
```

This isn't rate limiting in the Part 2 sense at all — there's no clock, no bucket, no window. It's a guard that says "no single run is allowed to fan out into more than 5 parallel branches, full stop, regardless of how much budget is left or how fast the clock is ticking." If an upstream planning step ever produced 500 queries instead of 5 — a prompt-injection payload, a model hallucinating a huge list, a bug — this is what catches it, and none of the time-based algorithms in Part 2 would have, because 500 requests arriving in one burst from one run doesn't necessarily look anomalous to a limiter that's watching aggregate traffic across all users.

The general pattern for any agentic system: put a hard ceiling on (a) fan-out width per step, (b) total steps/node-visits per run, and (c) total recursion depth for any planning/replanning loop. These are cheap to implement, catch an entire class of bug that temporal rate limiting structurally cannot see, and should be treated as a *required* companion to Part 2's limiters, not a redundant extra.

Part 5: the unknown-cost-upfront problem

An LLM call's true cost isn't known until it finishes — `max_tokens` is an upper bound, not the actual cost, and with streaming output the exact token count only exists once the stream ends. This breaks the simple "charge cost, then proceed" model from Part 2's token bucket cleanly.

Two practical answers, and they compose:

- **Charge an estimate upfront, reconcile after.** Admit the call against `cost = max_tokens / 1000` (or a cheaper heuristic on prompt length), then once the real usage is known, credit the difference back into the bucket. This means the bucket is briefly *pessimistic* (it holds less available capacity than it truly has, mid-flight) rather than optimistic — the safe direction to be wrong in.
- **Charge incrementally while streaming**, for cases where even a temporary over-reservation is unacceptable (a very tight per-second provider cap) — more accurate, but couples the limiter to the generation loop itself rather than just the request boundary, which is real added complexity to weigh against the accuracy gain.

The tool-call version of this problem is different, not the same problem restated. A tool call that costs real money per invocation (a paid third-party lookup) or carries real risk (a destructive action) isn't well modeled as "tokens" at all — the right budget there is denominated in **dollars or in a risk-weighted count**, tracked per tool, independent of the LLM token budget. Conflating the two — treating a $2-per-call paid API and a $0.001 LLM call as the same "unit" in one shared budget — hides the fact that ten calls to the expensive tool can quietly consume what was meant to be a whole day's spend.

Part 6: protecting fragile things downstream (the leaky-bucket case, concretely)

`send_tavily.py`'s `retrieve` node calls a third-party search API from inside a `Send`-based fan-out — meaning several branches can call out to Tavily concurrently, from a single run:

```
async def retrieve(state: SubState) -> dict:
    try:
        result = await tavily.search(query=state["query"], max_results=2, include_raw_content=False)
        ...
    except (UsageLimitExceededError, BadRequestError, ForbiddenError,
            InvalidAPIKeyError, TavilyTimeoutError, requests.exceptions.RequestException) as e:
        hits = [{"query": state["query"], "error": str(e)}]
    return {"hits": hits}
```

Two things worth naming precisely here:

- **The fan-out ceiling from Part 4 is doing double duty.** `MAX_QUERIES` doesn't just bound this run's own cost — it bounds how many concurrent requests get thrown at Tavily at once, which is a leaky-bucket-shaped concern (don't burst a downstream provider) being partially satisfied by a structural ceiling (Part 4) rather than a temporal limiter (Part 2). In a system with a much wider legitimate fan-out requirement, you'd want an actual leaky bucket in front of the Tavily client specifically, not just a fixed cap on branch count — the cap protects against pathological width; a real leaky bucket would also pace *legitimate* width against Tavily's own rate limit.
- **`UsageLimitExceededError` being caught per-branch, not per-run, is a deliberate isolation choice.** One branch hitting Tavily's own rate limit doesn't fail the whole run — it returns a hit annotated with an error, and `compile_answer` (elsewhere in the same file) already separates `good_hits` from `failed_queries` and reports both. This is the same per-branch failure isolation principle from `SYSTEM_DESIGN_PATTERNS.md`'s design 1, applied to the specific case where the failure *is* a rate-limit rejection from something downstream.

Part 7: rate limiting to protect a human, not just a system

An approval-gated write agent (see `SYSTEM_DESIGN_PATTERNS.md` design 3) doesn't remove the need for rate limiting — it relocates part of the problem. The *proposal-generation* step still needs standard LLM rate limiting. But there's a layer that's easy to miss entirely: **the approval queue itself has a capacity, and it's a human's.**

If an agent (or a burst of triggering events) generates proposals faster than a human reviewer can reasonably evaluate them, the queue backs up, review quality degrades under pressure, and the entire safety property the approval gate exists for gets quietly eroded — not because the gate failed, but because it was never paced to the reviewer's actual throughput. The fix is the same shape as everything else in this doc, applied to a human instead of a downstream system: a bounded queue with a defined max depth, and a policy for what happens when it's full (queue, reject new proposals, or — the leaky-bucket framing — smooth the *rate* at which new proposals are surfaced to the reviewer, even if the agent itself generated them in a burst).

Part 8: retries, backoff, and why a retry storm is a self-inflicted denial-of-wallet

A 429 response should trigger **exponential backoff with jitter** on the caller's side, not an immediate retry — a fleet of callers all retrying on the same fixed delay after being rejected synchronizes into another burst, arriving together and getting rejected together, indefinitely. Jitter (randomizing the delay slightly per caller) breaks that synchronization.

The sharper point for an agentic system specifically: **retries must be idempotent**, independent of rate limiting entirely — this is the same requirement covered in depth in `docs/CAP_THEOREM_AND_AGENTIC_SYSTEMS.md` for write actions, and it applies here for exactly the same reason. A naive retry loop that doesn't know whether its previous attempt actually succeeded or just timed out can turn a single legitimate call into several actual calls to whatever's downstream — which is, mechanically, a self-inflicted denial-of-wallet event, generated by your own retry logic rather than by an attacker or a misbehaving user.

Rapid-fire

Probe	One-line answer
Token bucket vs leaky bucket	Token bucket controls admission and allows bursts (protects you from a bursty caller); leaky bucket controls output pacing and smooths bursts away (protects whatever's downstream of you).
Why is `cost=1` per request wrong for LLM traffic	One call can be 200 tokens, another 200,000 — charge `cost = estimated_tokens / 1000` so the budget is a token budget, not a request-count budget.
Fixed window's flaw	No memory of the previous window — a client can get 2× the limit by straddling the boundary.
Sliding window log vs counter	Log is exact but O(limit) memory per key; counter is O(1) and approximate (assumes uniform distribution in the previous window) — counter is the production default at scale.
Why does an agentic system need more than one rate limiter	A single inbound request can fan out into unbounded internal calls — limits are needed at the user, per-run, per-tool-call, per-tool-type, and provider layers, each catching a different failure mode.
Why isn't a time-based rate limiter enough for a runaway planning loop	The loop can exhaust budget entirely within one run, in seconds — aggregate traffic can still look normal system-wide. Needs a structural ceiling on fan-out width / step count, independent of the clock.
How do you rate-limit a call whose cost isn't known until it finishes	Charge an estimate upfront (e.g. from `max_tokens`), reconcile the difference once real usage is known — stay pessimistic, not optimistic, while it's in flight.

Probe	One-line answer
Why can't a tool-call budget and an LLM token budget share one bucket	They're denominated in different things (dollars/risk vs. tokens) — combining them hides the fact that a few calls to an expensive tool can consume what was meant to be a whole day's LLM budget.
What does rate limiting protect in a HITL system that a human reviews	The reviewer's own throughput — an unbounded proposal queue erodes review quality even though the approval gate itself never technically failed.
Why is a naive retry-on-429 dangerous	Synchronized retries (no jitter) re-create the burst that got rejected; non-idempotent retries can turn one logical call into several real ones — a self-inflicted denial-of-wallet.

Source: `COMMAND_PATTERNS_REVISION.md`

LangGraph `Command` — Patterns Reference

Condensed from the full deep-dive session. Companion runnable scripts in this folder: `command.py` (routing), `retry_mechanism_with_command.py` (loops), `interrupt_command.py` (HITL basics), `approval_service.py` (prod-shaped HITL service).

The one idea everything else follows from

> State mutation and routing are usually the *same decision*, made by the *same code*, with the *same information*. `Command` lets a node express both atomically in one return value, instead of splitting it into a state write plus a separate router function that has to stay in sync by hand.

```
Command(update=dict, goto=str | list[str], graph=Command.PARENT, resume=Any)
```

Must-know patterns

1. State update + routing together

```
def check_amount(state) -> Command[Literal["manual_review", "auto_approve"]]:
    if state["amount"] > 10_000:
        return Command(update={"status": "needs_review"}, goto="manual_review")
    return Command(update={"status": "approved"}, goto="auto_approve")
```

Cleaner than `add_conditional_edges` when the branch condition *is* the node's own business logic — no separate router function re-deriving it.

2. Loops need a monotonic counter + a check `agent → execute_tool → agent` cycles are just ordinary edges; LangGraph doesn't special-case cycles. Without a field that moves toward a stop condition (`attempts`) and a node that checks it, you get infinite execution (backstopped only by `recursion_limit`, not a design strategy).

3. `interrupt()` + `Command(resume=...)` — human-in-the-loop

- `interrupt(payload)` freezes the graph, checkpoints full state under `thread_id`, and surfaces `payload` as `result["__interrupt__"]`.
- `graph.invoke(Command(resume=value), config={"configurable": {"thread_id": ...}})` reloads that checkpoint and makes the *same* `interrupt()` call return `value` instead of pausing.
- Requires a checkpointer (`MemorySaver` for dev, `SqliteSaver`/`PostgresSaver` for anything that must survive a restart).

4. THE gotcha: interrupted nodes restart from the top on resume Everything *before* `interrupt()` in that node re-runs on every resume.

```
# BAD — runs twice: once on first call, again on resume
def human_approval(state):
    send_slack_notification(...)      # ⚠️ non-idempotent, re-executed on resume
    decision = interrupt({...})
```

Fix: separate think → interrupt → act into different nodes. Nothing non-idempotent may sit before or inside the `interrupt()`-containing node.

5. Approval gate shape (production HITL)

```
prepare (pure, no side effects) → human_approval (interrupt only) → execute (side effect, runs once)
```

This is `approval_service.py`. The `resume` value in real systems comes from an actual HTTP request body / button click / Slack payload — not a hardcoded string. Two separate endpoints: one that starts the graph and returns the interrupt payload, one that receives the decision and resumes.

6. Static edge + Command trap (classic bug)

```
builder.add_edge("router", "A")          # ⚠️ still fires unconditionally
def router(state) -> Command[Literal["B"]]:
    return Command(goto="B")             # BOTH A and B execute — no error, just silently wrong
```

`Command(goto=...)` **adds** a dynamic edge; it does not cancel a statically declared one. Rule: a node that ever returns `Command(goto=...)` must have zero `add_edge` calls registering its outgoing edges.

7. Fan-out: `goto=[...]` vs `Send`

- `Command(goto=["A", "B"])` — fixed node names, same state, scheduled for the same superstep, no ordering guarantee.
- `Command(goto=[Send("worker", {...}), Send("worker", {...})])` — same node invoked N times with **different, per-item** input. Needed for "map over a dynamically-sized list" (map-reduce).
- Either way: if parallel branches write the same state key, you need a reducer (`Annotated[list, operator.add]`) or you get `InvalidUpdateError`.

8. `Command.PARENT` — escaping a subgraph Inside a subgraph, plain `goto="x"` can only reach that subgraph's own nodes. `Command(goto="parent_node", graph=Command.PARENT)` routes to a node in whichever graph is hosting this one — used for specialist → supervisor escalation, or child security-check → parent human-review handoff.

Comparison table (compressed)

	`add_edge`	`add_conditional_edges`	`Command(goto=)`	`Command(update=,goto=)`	`Send`	`interrupt` + `resume`
Routing decided by	builder (static)	router fn	node itself	node itself	node itself	resuming caller
Changes state?	no	no	no	yes	yes	yes
Parallel?	only if multi-edge	if router returns list	if goto is list	if goto is list	yes, natively	n/a
Typical use	fixed pipeline	pure branch, no state	pure dispatch	classify-and-act	map-reduce	approval gates

Decision tree

```
Need to decide WHERE to go, based on something just computed?
├─ No  → add_edge / add_conditional_edges
└─ Yes → Also need to write state as part of it?
         ├─ No  → Command(goto=...)
         └─ Yes → Per-item dynamic fan-out (map over a list)?
                  ├─ Yes → Command(goto=[Send(...), ...])
                  └─ No  → Need to pause for a human/external event?
                           ├─ Yes → interrupt() + Command(resume=...)
                           └─ No  → Destination outside this subgraph?
                                    ├─ Yes → Command(goto=..., graph=Command.PARENT)
                                    └─ No  → Command(update=..., goto=...)
```

Self-test questions

1. Why does `Command` exist when conditional edges already support dynamic routing?
2. A node returns `Command(goto="B")` but a static `add_edge` to `A` also exists from that node. What actually runs?
3. Exactly what re-executes when a graph resumes after `interrupt()`?
4. When do you need `Send` instead of `Command(goto=[...])`?
5. Two parallel branches write the same non-`Annotated` state key — what happens at runtime?
6. Why should specialists in a supervisor architecture hand back to the supervisor instead of calling each other directly?
7. How do you stop an LLM-invoked tool from routing straight into a destructive action with no human check?

Debugging scenarios

- **"Both branches always run"** → leftover `add_edge` alongside a `Command(goto=...)` node (Pattern 6 above).
- **"User got charged twice"** → non-idempotent side effect placed before or inside the `interrupt()`-containing node (Pattern 4 above).
- **`InvalidUpdateError` only under real load, never in single-item tests** → missing reducer on a field multiple `Send`-spawned workers all write to (Pattern 7 above).

One-line whiteboard summary

`Command` is the wire that lets one function emit both "what should state look like now" and "where do we go from here" from a single evaluation, instead of forcing you to write that decision twice and hope the two versions never disagree.
